# Which operations in numpy uses SIMD?

Asked 1 year, 8 months ago Modified 1 year, 8 months ago Viewed 681 times

2

Good afternoon! Currently, I'm diggin out the reason why numpy is fast. More specific, I'm wondering why np.sum() is that fast. My one suggestion is np.sum() uses some kind of SIMD optimization, but I'm not sure whether it is. Is there any way that I can check which numpy's method uses SIMD operations? Thx in advance

numpy simd

Share Improve this question Follow

asked Feb 10, 2022 at 2:25
happychild
93 7

1 Fast compared to what? Compared to python operations, such as on lists, the primary reason is that python is interpreted with lots of `object` lookup, while most of the `numpy` methods used compiled code. Or are you comparing `numpy` to your own C or C++ code? – hpaulj Feb 10, 2022 at 2:29

Currently I'm comparing w/ my cpp code, which uses direct access to pointer. Could you be more elaborate on "numpy's compiled code" part? I took quite long time on understanding numpy's github and documentation, but I didn' reach to adequate point.. – happychild Feb 10, 2022 at 2:32

Navigating the numpy code is complicated, and I've only dabbled in it. I believe the use of SIMD and such will be more a function of the compiler, or maybe some lower level routines. Note that `np.sum` passes the task to `np.add.reduce`, which means it takes advantage of the whole `ufunc` machinery. There's another recent question about `np.sum`, stackoverflow.com/questions/71056235/internals-of-numpy-sum – hpaulj Feb 10, 2022 at 3:09

Sum of an array is a classic use-case for SIMD; I'd be very disappointed if NumPy didn't have a SIMD implementation of it, and would defeat part of the purpose of NumPy. We can tell that NumPy is at least using multiple accumulators, hopefully across multiple SIMD vectors, for FP sums, because it rounds differently from the worst-case naive `sum += a[i]`: Simd matmul program gives different numerical results / How to avoid less precise sum for numpy-arrays with multiple columns – Peter Cordes Feb 10, 2022 at 3:14

1 @PeterCordes I was disappointed too when I saw this was not vectorized. Hopefully, SIMD instruction are often used by Numpy for basic operators other than reduction (currently). – Jérôme Richard Feb 10, 2022 at 13:20

## 1 Answer

Sorted by: Highest score (default)

6

Numpy does not currently use SIMD instructions for trivial `np.sum` calls yet. However, I made this PR which should be merged soon and fix this issue with integers (it will use the 256-bit AVX2 instruction set if available and the 128-bit SSE/Neon instruction set otherwise). Using SIMD instructions for `np.sum` with floating-point numbers is a bit harder due to the current algorithm used (pair-wise summation) and because one should care about the precision.

> Is there any way that I can check which numpy's method uses SIMD operations?

Low-level profilers and hardware-counter-based tools (eg. Linux perf, Intel VTune) can do that but they are not very user-friendly (ie. you need to have some notions in assembly, know roughly how processors work and read some documentation about hardware counters). Another solution is to look the disassembled code of Numpy using tools like `objdump` (require a pretty good knowledge in assembly and the name of the C function called) or simply look at the Numpy C code (note compilers can autovectorize loops so this solution is not so simple).

**Update:** If you are using `np.sum` on *contiguous double-precision* Numpy arrays, then the benefit of using SIMD instructions is

**Join Stack Overflow** to find the best answer to your technical question, help others answer theirs.

Sign up with email Sign up with Google Sign up with GitHub Sign up with Facebook

For contiguous medium-sized arrays (eg. >10K and <1M items), using SIMD instructions should result in a significant speed up, especially for simple-precision arrays (eg. 3-4 times faster on DP and 6-8 times faster on SP on mainstream machines).

Share Improve this answer Follow

edited Feb 12, 2022 at 0:17

answered Feb 10, 2022 at 13:16

Jérôme Richard
**42.9k** 6 30 62

Oh, it does pairwise summation? On average, it doesn't matter which elements you pair together, so you can do pairs *of vectors*, i.e. do the normal pairwise-summation algorithm, but treating it as an array of SIMD vectors, with an hsum at the end. This still avoids adding 1 number to the sum of many numbers. This wouldn't be deterministic across different vector lengths, unless you make the scalar fallback and 16-byte vector versions pair far enough away to leave room for growth (to at least 1024-bit vectors = 32 floats) by choosing how you pair in an unrolled loop. – Peter Cordes Feb 10, 2022 at 13:40

1 @PeterCordes Yeah. I check the code a bit. It turns out they already made a partial unrolling optimization. Besides a small performance issue (related to strides), the main problem is that GCC appear to simply fail to autovectorize the partially unrolled loop... Removing prefetching instructions in the loop cause GCC to actually vectorize the loop but this is finally surprisingly slower with the vectorized code! The asm code contains many conditional jumps but they should be predicted in my tests. In fact, `perf` reports they are quite cheap. – Jérôme Richard Feb 11, 2022 at 10:31

@PeterCordes In the code, they commented that "small blocksizes seems to mess with hardware prefetch" so they use software prefetching. It looks like something is really wrong with the hardware-prefetcher/caches in this code but I do not understand why yet. I may be related to the pair-wise reduction... In the end, the current implementation on float64 arrays is already fast enough to saturate the DRAM bandwidth on my machine (but not the caches). It is still quite good due to the 2 `addsd` being executed in parallel and completely pipelined resulting in a 16 byte/cycle throughput. – Jérôme Richard Feb 11, 2022 at 10:38

It's really a lot dipper than I thought... Thx for your recommendations, and I will quote out some points if I get something – happychild Feb 12, 2022 at 12:33

**Join Stack Overflow** to find the best answer to your technical question, help others answer theirs.

Sign up with email